

SOFTNIX SOLUTIONS — ROUND 2 BACKEND

Technical Assessment Guidelines

Welcome to Round 2 of the Softnix Intern Drive. This assessment evaluates your ability to build robust, secure, and well-structured backend systems using modern Java backend standards.

SUBMISSION DEADLINE

- **Date & Time:** August 30, 2026, by 6:00 PM IST
 - **Project Review** will commence on August 31, 2026, at 10:00 AM IST
 - *Late submissions will not be considered.*
-

MANDATORY TECH STACK & ARCHITECTURE

Your implementation must strictly adhere to the following tools and practices:

1. Language: Java 21

2. Framework: Spring Boot

3. Database: MySQL or PostgreSQL

- Use proper JPA entity relationships (@OneToMany, @ManyToOne, etc. where applicable) — no flat, denormalized tables.
- Create the tables listed in your individual brief at minimum; add any supporting tables your design needs.

4. API Design

- RESTful endpoints following standard HTTP verbs and status codes.
- Layered architecture: Controller → Service → Repository. No business logic inside controllers.
- Centralized exception handling (@ControllerAdvice) with clear, consistent error responses.

5. Validation

- Use Bean Validation (@Valid, @NotNull, @Min, etc.) on request DTOs.
- Every business rule listed in your assigned brief must be enforced server-side, not just assumed by the client.

6. Build Tool: Maven

7. ORM: Spring Data JPA

8. API Testing Tool: Postman

- A working Postman collection covering all endpoints must be submitted alongside your code.

9. Version Control & GitHub Hygiene

- Initialize a public GitHub repository.

- Maintain a clear commit history using conventional commit messages (feat:, fix:, refactor:, docs:).

10. Live Deployment (Optional)

- Hosted on Render, Railway, or a similar platform, if attempted.
-

BONUS POINTS (OPTIONAL)

- **API Documentation:** Swagger / OpenAPI (springdoc-openapi).
 - **Containerization:** A working Dockerfile for the application.
 - **Logging & Error Handling:** Structured logging (SLF4J) alongside centralized exception handling.
 - **Caching:** Spring Cache or Redis for frequently read data.
-

ASSIGNED PROJECT BRIEF

- Your specific project scenario, business rules, and required database tables have been shared with you as an individual Technical Specification & Project Brief.
 - Please build your solution around the exact business rules, entities, and core functionality specified in your individual brief.
 - The tables listed in your brief are the minimum required — create any additional tables, join tables, or columns needed to correctly implement the business rules.
 - **We are welcoming innovations** — priority is given to candidates who initiate innovation in their task (e.g., thoughtful edge-case handling, clean validation messages, useful extra read APIs).
-

README.md REQUIREMENTS

Your GitHub repository must include a clear README.md containing:

1. **Project Title & Overview:** a short description of your assigned system.
 2. **Setup Instructions:** mvn clean install, mvn spring-boot:run (or the equivalent for your build tool).
 3. **Database Setup:** schema/connection details (application.properties or application.yml placeholders).
 4. **API Testing Instructions:** how to import and run the Postman collection.
 5. **Test Instructions** (if implemented): mvn test.
 6. **Live Demo Link** (if deployed).
-

SUBMISSION PROTOCOL

Submit your project before **August 30, 2026, by 6:00 PM IST** to **devsoftnix@gmail.com**:

- Full Name
- Assigned Project Title
- Public GitHub Repository URL
- Live Deployment URL (optional)

Best of Luck!